SkillSync

Intelligent Career Mentor & Learning Analytics Platform

A Project Report Submitted in Partial Fulfilment of Academic Requirements

By

Gayatri Harilal Yadav

Index

Acknowledgement

The project "SkillSync: Intelligent Career Mentor & Learning Analytics Platform" is the project work carried out by Gayatri Harilal Yadav under the guidance of my project mentor.

I am thankful to my project guide and faculty mentors for their continuous support in completing this project. Their suggestions and valuable technical inputs regarding system design, REST API configuration, and project report formatting provided immense assistance throughout the project lifecycle.

I am also deeply thankful to my family members and friends who were always there to provide moral support, motivation, and active testing feedback during development.

Abstract

The project called SkillSync is an intelligent web application engineered to assess student technical competencies quickly, reliably, and with actionable career insights. Traditional academic grading systems assign static overall scores without isolating the underlying technical deficits behind them. SkillSync bridges this gap by providing an end-to-end full-stack platform where students undergo multi-domain diagnostic assessments across Web Development, Backend & Databases, Data Science & AI, and Cloud & DevOps.

The platform features automated role recommendation (mapping students to roles such as Full-Stack Engineer or QA Associate), real-time domain breakdown charts, and an algorithmic Action Plan engine. The Action Plan identifies a learner's lowest-scoring domain and recommends a targeted, project-based rectification roadmap with focus areas and estimated time-to-fix.

A comprehensive gamification ecosystem — featuring live Experience Points (XP), dynamic rank-tier upgrades, active streak counters, milestone task checklists, and achievement badges — drives sustained learner engagement. Direct integration with leading tech recruitment portals allows learners to align their verified capabilities with active market demand.

1. Introduction

In today's age of Information Communication and Technology (ICT), web applications and intelligent assessment platforms play an essential role in remote education and technical workforce preparation. The software industry is expanding rapidly, demanding proficiency in specialised domains such as cloud infrastructure, full-stack development, and artificial intelligence.

Students often struggle to determine whether their technical knowledge satisfies current hiring benchmarks. SkillSync is designed as a dynamic, AI-assisted mentor platform that evaluates learner competencies in real time, provides transparent analytics, and delivers structured guidance on what to learn next.

1.1 Objective of the Present Work

- To develop a modern, responsive, and secure full-stack web platform for automated technical skill diagnostics.
- To provide multi-domain assessments covering Web Development, Databases, AI/Data Science, and Cloud Systems.
- To dynamically calculate Industry Readiness Scores and map evaluated learners to suitable career roles.
- To build an algorithmic Gap Analysis and Action Plan engine that isolates competency deficits and provides step-by-step project roadmaps.
- To implement gamified progression metrics, including XP points, rank elevations, active learning streaks, and milestone badges.
- To connect evaluated student talent directly with global technology recruitment gateways.

3. System Analysis

3.1 Problem Definition

Many online learning platforms provide generic scores without explaining the specific domain strengths and weaknesses of a student. Learners are often left confused regarding exactly which technical skills they lack and what projects they must build to close those gaps. Traditional manual evaluation also requires significant instructor bandwidth and does not scale.

SkillSync solves this problem with an automated diagnostic engine. It evaluates question-level responses across multiple domains, detects the weakest technical discipline, and generates an immediate remediation roadmap complete with focus areas, a practical project blueprint, and an estimated completion timeline.

3.2 Preliminary Investigation

- Purpose: SkillSync is designed to deliver accessible, personalised career mentorship through a web platform, replacing static evaluations with dynamic learning diagnostics.
- Benefits: Instant skill analysis, automated domain gap detection, gamified learning incentives, and direct industry recruitment portal integration.
- Proposed System: Structured multi-domain evaluation, automated role mapping, interactive visual charts, and actionable rectification paths.

3.3 Feasibility Study

- Technical Feasibility: Built using Python (FastAPI framework), SQLite/MySQL database, SQLAlchemy ORM, HTML5, and Tailwind CSS. The stack is lightweight, standards-compliant, and runs seamlessly across modern browsers.
- Economic Feasibility: The platform relies entirely on open-source frameworks, avoiding proprietary licensing fees and keeping infrastructure overhead minimal.
- Operational Feasibility: The interface provides intuitive navigation, visual data charts, and requires zero installation on client devices.
- Schedule Feasibility: Development was executed across structured sprint intervals, ensuring the authentication, assessment, analytics, and gamification modules were all delivered on schedule.
- Social Feasibility: The platform promotes self-directed, judgement-free learning by pairing every weak score with a constructive next step rather than a discouraging label.

3.4 Software Requirement Specification (SRS)

Hardware Requirements

- Processor: Intel Core i3 / i5 or equivalent
- RAM: Minimum 4 GB (8 GB recommended)

- Storage: At least 500 MB free space
- Connectivity: Internet access for live API calls and portal integration

Software Requirements

- Operating System: Windows 10/11 or Linux
- Backend: Python 3.10+, FastAPI, Uvicorn (ASGI server)
- Database: SQLite3 (development) / MySQL (production), SQLAlchemy ORM
- Frontend: HTML5, Tailwind CSS, JavaScript, Chart.js
- Browser: Google Chrome, Microsoft Edge, or Mozilla Firefox (latest versions)

3.5 Software Engineering Paradigm

The project follows an Adapted Iterative Waterfall Model. Development proceeds through clearly defined phases, but feedback loops between testing and implementation allow refinements to be folded back into earlier stages — particularly useful for tuning the scoring engine and gamification balance based on real testing sessions.

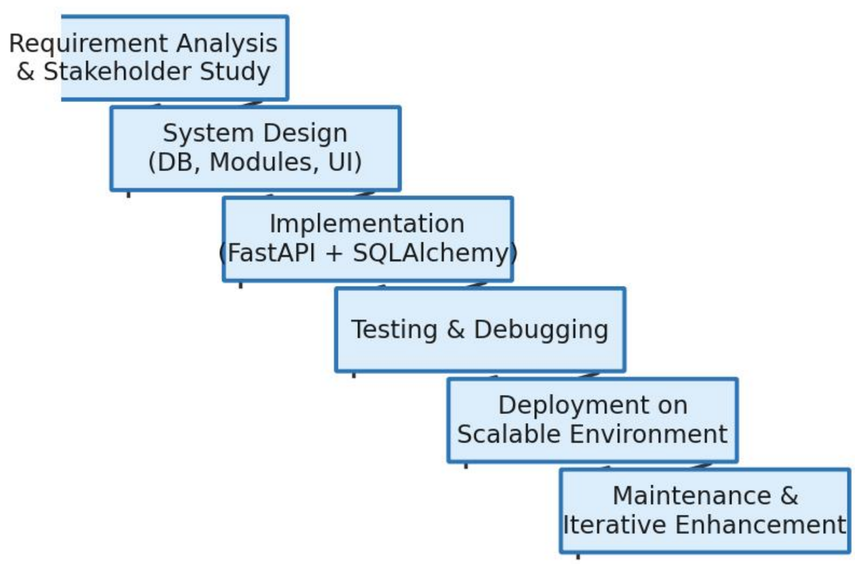

Fig. 3.1: Adapted Iterative Waterfall Model followed for SkillSync development

4. System Design & Data Flow Diagrams

A Data Flow Diagram (DFD) is a graphical representation of how data moves through the SkillSync platform — what transforms it, where it is stored, and which external entities interact with the system. DFDs are presented here across three levels of increasing detail, followed by the Entity-Relationship model of the underlying database.

4.1 Level 0 DFD (Context Diagram)

The context-level DFD shows the SkillSync Engine as a single process exchanging data with four external entities: the Student/Learner, the Administrator, the Question Bank, and partner Industry Hiring Portals.

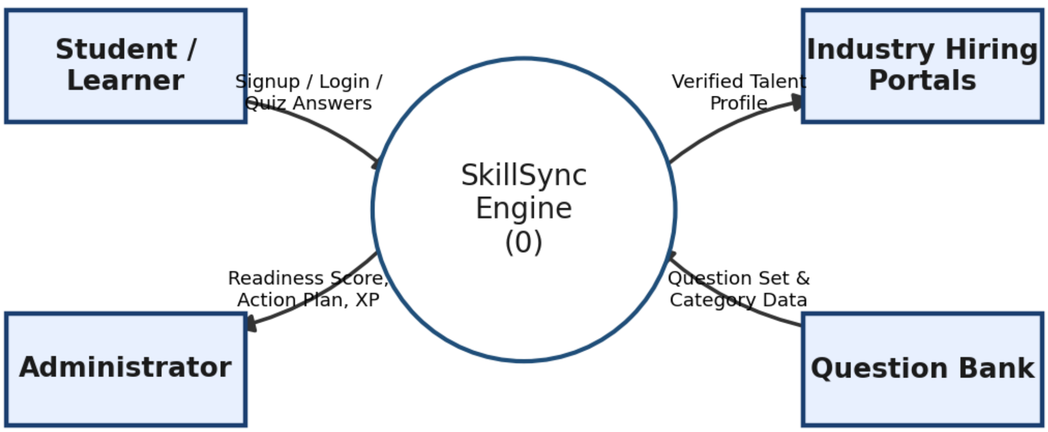

Fig. 4.1: Level 0 (Context) DFD of the SkillSync Engine

4.2 Level 1 DFD

The Level 1 DFD decomposes the SkillSync Engine into its eight core functional modules — Authentication, Quiz Evaluation, Gap Analytics, Gamification, Action Plan Generation, Portal Gateway, Student Profile Management, and Admin & Question Bank Management — each of which exchanges data bidirectionally with the core engine.

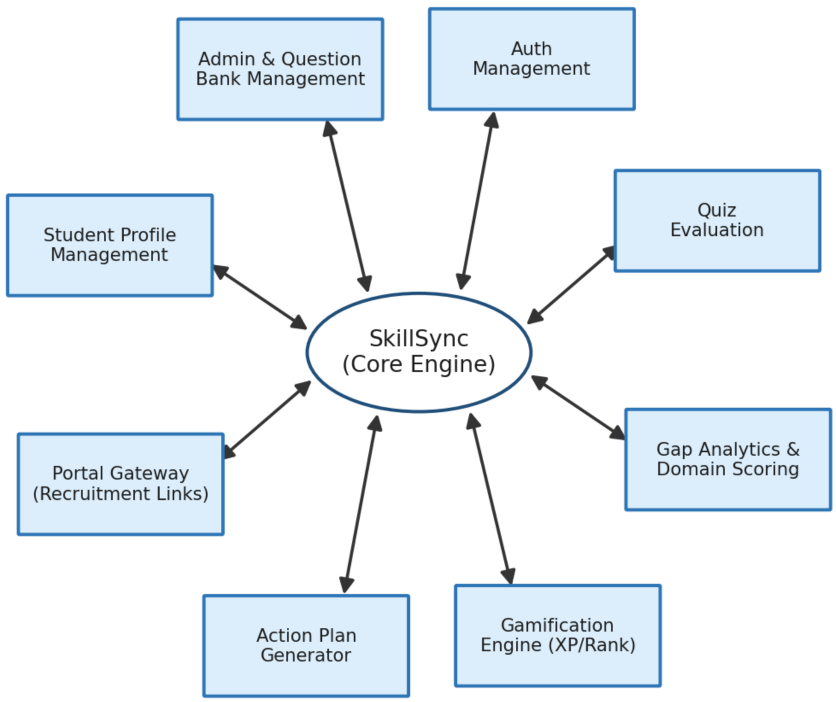

Fig. 4.2: Level 1 DFD showing the eight core modules of SkillSync

4.3 Level 2 DFD — Quiz Submission & Scoring Process

This second-level DFD traces the detailed internal workflow the moment a student submits an assessment: credential verification, question retrieval from the Question Bank, per-domain scoring, XP/rank updates, action-plan generation, and final display on the analytics dashboard.

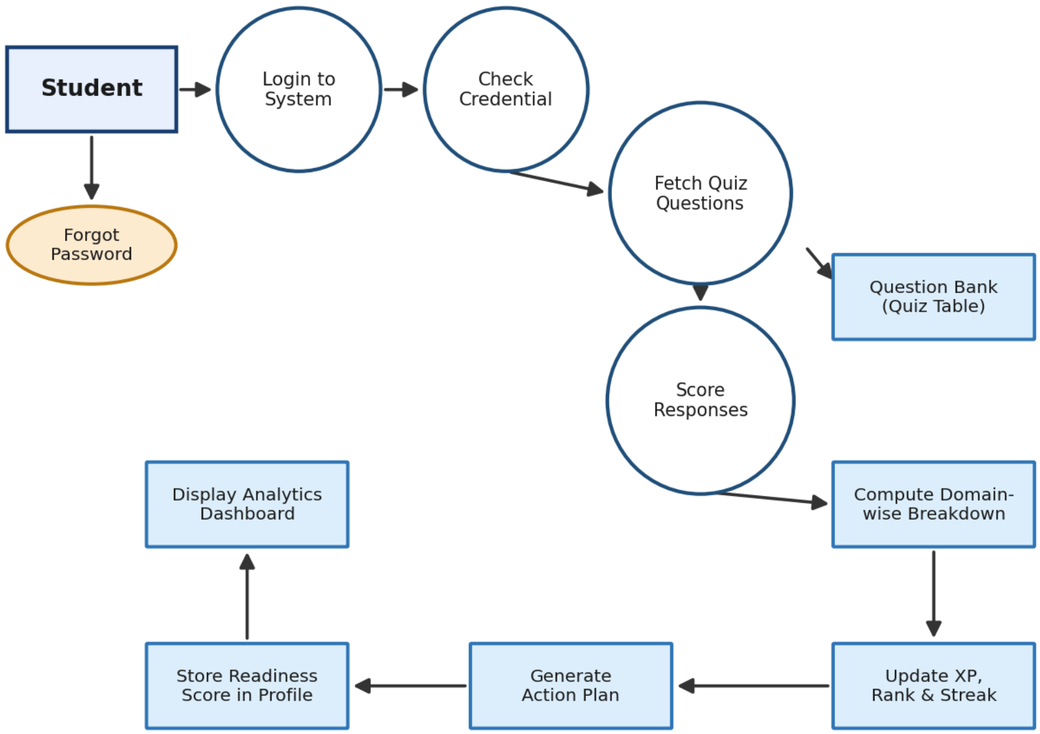

Fig. 4.3: Level 2 DFD — detailed quiz submission and scoring workflow

4.4 Entity-Relationship (ER) Diagram

The ER diagram models the six core entities of the SkillSync database — Student, StudentProfile, Gamification, QuizQuestion, ActionPlan, and AdminUser — along with their attributes and the relationships that connect them.

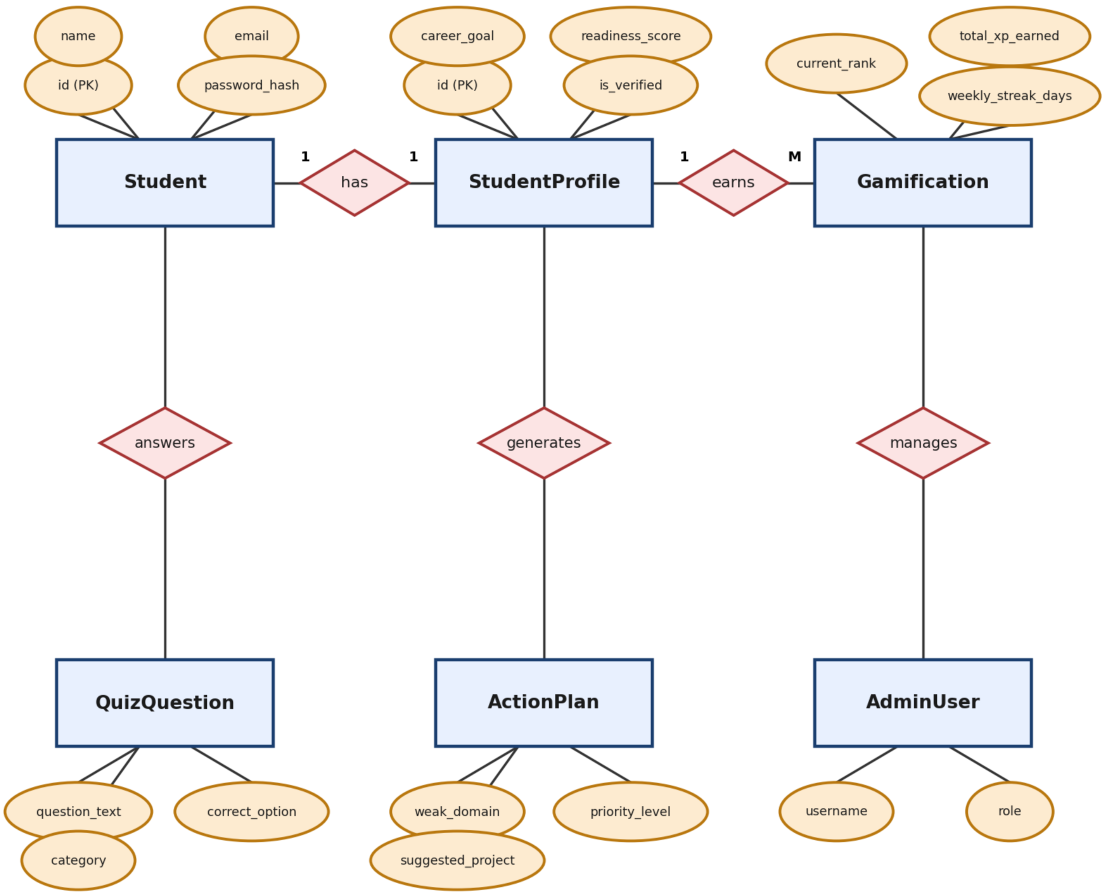

Fig. 4.4: Entity-Relationship Diagram of the SkillSync database

5. Data Structure of All Modules

SkillSync manages its relational entities using SQLite/MySQL database models powered by SQLAlchemy ORM. The primary tables are described below.

Table 1: Student (Authentication & Profile Record)

Field	Type	Null	Key	Extra
id	INTEGER	NO	PRI	auto_increment
name	VARCHAR(150)	NO	–	–
email	VARCHAR(254)	NO	UNI	–
password	VARCHAR(255)	NO	–	Bcrypt Hashed
created_at	DATETIME	YES	–	current_timestamp

Table 2: StudentProfile (Readiness & Role Mapping)

Field	Type	Null	Key	Extra
id	INTEGER	NO	PRI	auto_increment
student_id	INTEGER	NO	MUL	FK → Student(id)
career_goal	VARCHAR(200)	YES	–	–
readiness_score	FLOAT	NO	–	0.0 to 100.0
is_verified	INTEGER	NO	–	Default 1

Table 3: Gamification (XP & Badges Ecosystem)

Field	Type	Null	Key	Extra
id	INTEGER	NO	PRI	auto_increment
student_id	INTEGER	NO	MUL	FK → Student(id)
current_rank	VARCHAR(100)	NO	–	Rank Tier
total_xp_earned	INTEGER	NO	–	Experience Points
weekly_streak_days	INTEGER	NO	–	Streak Counter

Table 4: QuizQuestion (Assessment Bank)

Field	Type	Null	Key	Extra
id	INTEGER	NO	PRI	auto_increment
question_text	VARCHAR(500)	NO	–	–
option_a–option_d	VARCHAR(200)	NO	–	Four choices
correct_option	VARCHAR(10)	NO	–	–
category	VARCHAR(100)	NO	–	Default: Web Development

6. Procedural Design (Workflow Flowcharts)

The procedural design illustrates the sequential logic executed by each type of user — from initial authentication through quiz completion, analytics review, and milestone execution.

6.1 Student Assessment Flow

Once logged in, a student lands directly on the analytics dashboard and can launch the diagnostic quiz. The system loops on unanswered questions until the assessment is complete, then submits it for scoring.

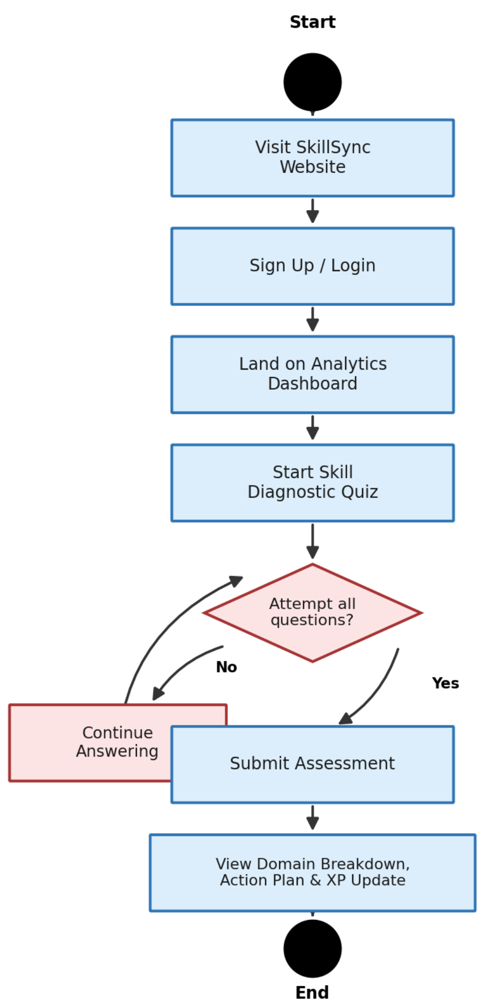

Fig. 6.1: Student assessment workflow, from login to results

6.2 Authentication Flow

Login credentials are verified using bcrypt password hashing. A successful match creates a session and an auth token before redirecting the user to their role-specific dashboard; a failed match returns an inline error and allows the user to retry.

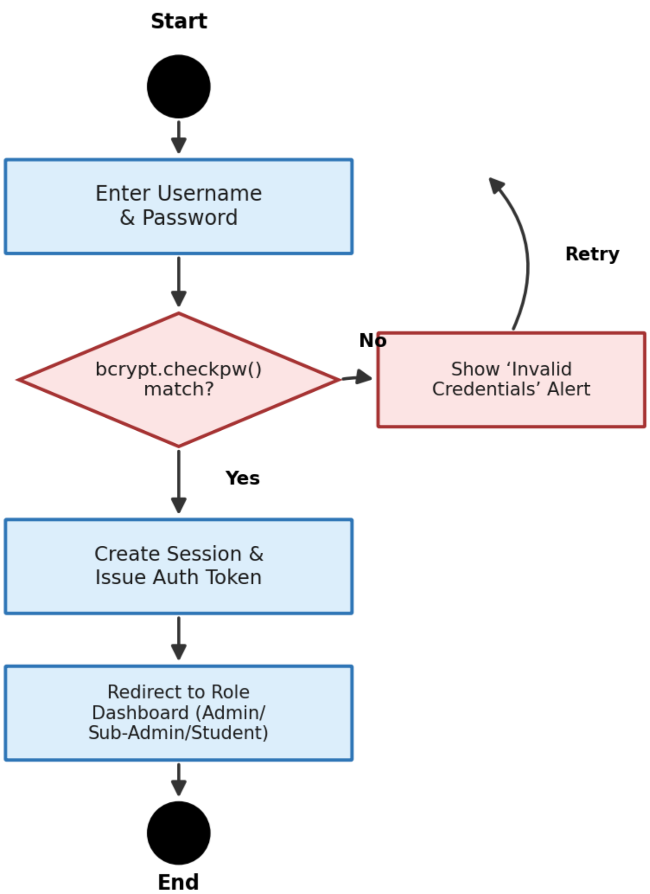

Fig. 6.2: Secure authentication workflow using bcrypt credential verification

7. System UI Screenshots

7.1 Intelligent Learning Analytics Dashboard

The SkillSync Dashboard displays the student's target role, total accumulated XP, readiness score percentage, market hiring demand, a dynamic domain-breakdown chart, and the interactive action plan.

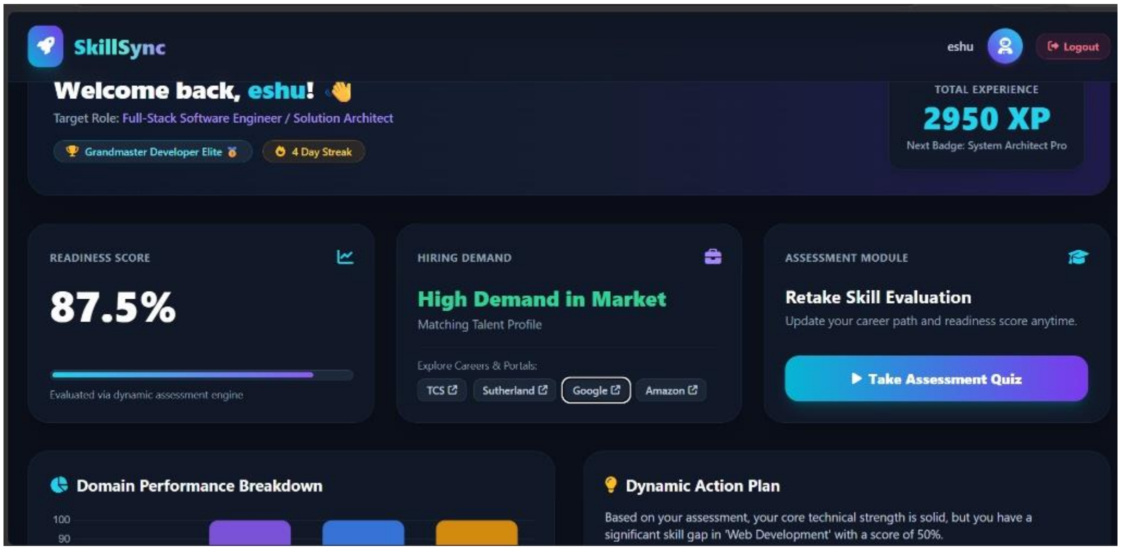

Fig. 7.1: SkillSync Analytics Dashboard with domain breakdown and target role

7.2 Dynamic Action Plan & Skill Gap Rectification Card

The Dynamic Action Plan card isolates the learner's weakest technical domain and prescribes a practical rectification project, key focus areas, priority level, and an estimated time-to-fix.

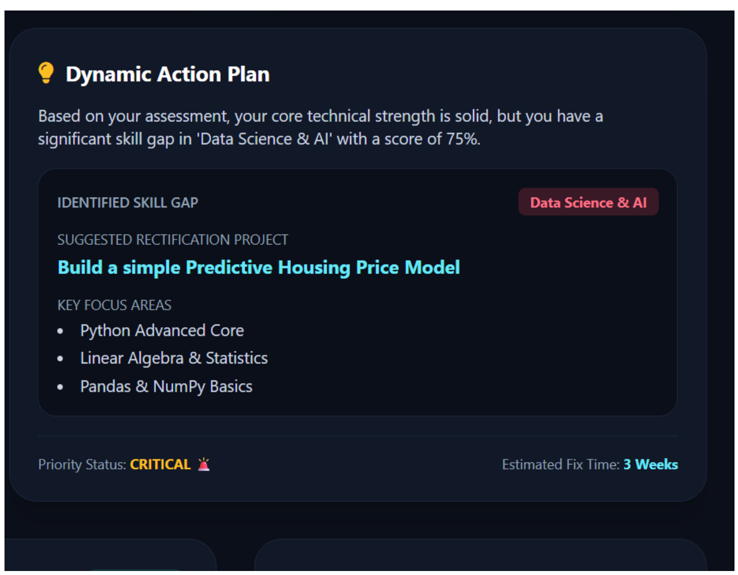

Fig. 7.2: Dynamic Action Plan with key focus topics and rectification project

7.3 Secure Authentication UI (Sign Up & Login)

The authentication interface features seamless tab switching between registration and login, with real-time field validation.

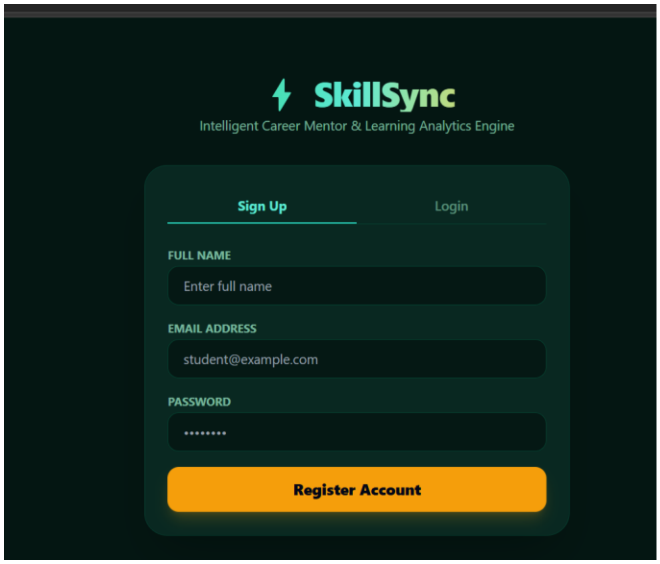

Fig. 7.3: Unified authentication view (Sign Up / Login tab switcher)

8. System Implementation & Source Code

8.1 Database Configuration & ORM Models (models.py)

```
from sqlalchemy import Column, Integer, String, Float, ForeignKey, DateTime
from sqlalchemy.orm import relationship
from datetime import datetime
from database import Base

class Student(Base):
    __tablename__ = "students"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(100), nullable=False)
    email = Column(String(100), unique=True, index=True, nullable=False)
    password = Column(String(255), nullable=False)
    created_at = Column(DateTime, default=datetime.utcnow)

class StudentProfile(Base):
    __tablename__ = "student_profiles"
    id = Column(Integer, primary_key=True, index=True)
    student_id = Column(Integer, ForeignKey("students.id"), nullable=False)
    career_goal = Column(String(150), default="Not Set Yet")
    readiness_score = Column(Float, default=0.0)
    is_verified = Column(Integer, default=1)

class Gamification(Base):
    __tablename__ = "gamification"
    id = Column(Integer, primary_key=True, index=True)
    student_id = Column(Integer, ForeignKey("students.id"), nullable=False)
    current_rank = Column(String(100), default="Novice Apprentice")
    total_xp_earned = Column(Integer, default=0)
    weekly_streak_days = Column(Integer, default=1)

class QuizQuestion(Base):
    __tablename__ = "quiz_questions"
    id = Column(Integer, primary_key=True, index=True)
    question_text = Column(String(500), nullable=False)
    option_a = Column(String(200), nullable=False)
    option_b = Column(String(200), nullable=False)
    option_c = Column(String(200), nullable=False)
    option_d = Column(String(200), nullable=False)
    correct_option = Column(String(10), nullable=False)
    category = Column(String(100), default="Web Development")
```

8.2 Core FastAPI Controller & Scoring Engine (main.py)

```
import bcrypt
from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.middleware.cors import CORSMiddleware
from sqlalchemy.orm import Session
from database import engine, Base, get_db
import models, schemas

app = FastAPI(title="SkillSync Engine", version="1.0.0")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], allow_credentials=True,
    allow_methods=["*"], allow_headers=["*"],
)

def hash_password(password: str) -> str:
    return bcrypt.hashpw(password.encode("utf-8"), bcrypt.gensalt()).decode("utf-8")
```

```
def verify_password(plain_password: str, hashed_password: str) -> bool:
    return bcrypt.checkpw(plain_password.encode("utf-8"), hashed_password.encode("utf-8"))

@app.post("/api/quiz/submit")
def submit_quiz(data: dict, student_id: int = 1, db: Session = Depends(get_db)):
    sid = data.get("student_id", student_id)
    answers = data.get("answers", {})
    category_correct, category_total = {}, {}
    total_questions, correct_answers = 0, 0

    for q_id, selected_option in answers.items():
        question = db.query(models.QuizQuestion).filter(
            models.QuizQuestion.id == int(q_id)).first()
        if question:
            total_questions += 1
            cat = getattr(question, "category", "Web Development")
            category_total[cat] = category_total.get(cat, 0) + 1
            if question.correct_option.strip().upper() == selected_option.strip().upper():
                correct_answers += 1
                category_correct[cat] = category_correct.get(cat, 0) + 1

    readiness = round((correct_answers / total_questions) * 100, 1) if total_questions > 0 else 0.0
    role = "Full-Stack Software Engineer / Solution Architect" if readiness >= 80 else "Junior QA
Associate"

    profile = db.query(models.StudentProfile).filter(
        models.StudentProfile.student_id == sid).first()
    if profile:
        profile.career_goal = role
        profile.readiness_score = readiness
        db.commit()

    return {"status": "Success", "industry_readiness_score": f"{readiness}%", "recommended_role":
role}
```

9. Software Testing & Quality Assurance

- Unit Testing: Verified cryptographic hashing (bcrypt) and readiness-score arithmetic against isolated test assertions for each function.
- Module Testing: Validated API endpoints for registration, authentication, quiz submission, and action-plan generation independently.
- Sub-system Testing: Combined the authentication, quiz-evaluation, and gamification modules to confirm they operate correctly end-to-end.
- System Testing: Verified the complete data flow from frontend form submissions through to database storage and dashboard rendering.
- Acceptance Testing: Tested the complete user journey across multiple browser viewports (desktop and mobile) to confirm real-world usability.

10. Report & Performance Analytics

Beyond individual dashboards, SkillSync aggregates readiness scores across all registered learners to give administrators a cohort-level view of where students are collectively strongest and weakest. This report guides curriculum and content-partnership decisions — for instance, a persistently low Data Science & AI average signals where new practice questions or workshops should be prioritised.

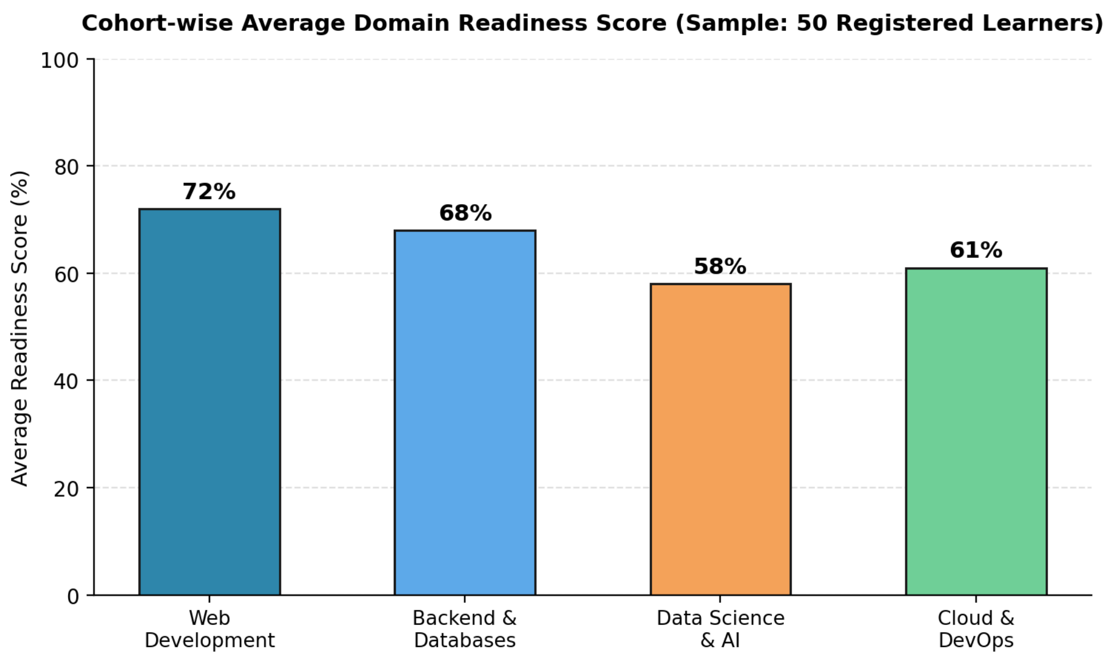

Fig. 10.1: Cohort-wise average domain readiness score, aggregated from all quiz submissions

The chart above reflects a sample cohort of 50 registered learners evaluated during the project's testing phase. Web Development scored highest on average, while Data Science & AI emerged as the most common skill gap — consistent with the individual Action Plan recommendations most frequently generated by the engine.

11. Future Scope & Conclusion

11.1 Future Scope

- Integrating LLM-based code evaluation for open-ended coding questions rather than multiple-choice only.
- Building native mobile apps for Android and iOS to widen accessibility.
- Enabling recruiter-facing search filters so hiring partners can directly query verified talent by domain and readiness score.
- Adding peer-benchmarking so learners can see how their readiness score compares anonymously against their cohort.
- Expanding the gamification system with team-based challenges and leaderboards.

11.2 Conclusion

SkillSync delivers an automated, intelligent career-mentoring platform that empowers students to identify and bridge their technical skill gaps effectively. By replacing static, generic scoring with domain-level diagnostics, a targeted Action Plan engine, and a genuinely motivating gamification layer, the system turns self-assessment into a clear, actionable roadmap. Designed with a lightweight, open-source stack and a modular FastAPI backend, SkillSync is well-positioned to scale toward richer assessment types, deeper analytics, and direct integration with the hiring pipeline.

12. Bibliography & References

- FastAPI Documentation — https://fastapi.tiangolo.com/
- SQLAlchemy Documentation — https://docs.sqlalchemy.org/
- Tailwind CSS Framework — https://tailwindcss.com/
- Chart.js Documentation — https://www.chartjs.org/
- Python Official Documentation — https://www.python.org/doc
- MySQL Official Site — https://www.mysql.com/